

Unit 07: Data Types and Operators

CONTENTS

Objectives

Introduction

7.1 Data Types

7.2 Additional Data Types

7.3 Operators

7.4 Relational Operators

7.5 Operator Precedence and Associativity

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Various Data types
- Explain arithmetic operators
- Describe conditional, logical and relational operators
- Describe arithmetic expression

Introduction

The term "data types" refers to a comprehensive framework for defining variables and functions of various types. The type of a variable decides how much storage space it takes up and how the stored bit pattern is interpreted. A value's data form identifies a set of properties that it shares.

An expression is a set of constants, variables, and operators that follow the grammatical rules of the language C and evaluate to a valid value. Activity refers to the impact that operators have on their operands.

Valid or well-formed expression refers to an expression that follows the grammar rules of the C programming language. An expression that is true or well evaluates to a single value of a valid C data form. C expression can be of the following types:

1. Numerical expressions always evaluates to a numeric value on which arithmetic operations can be performed. They can be further divided into the following two categories:

(a) Integer expression: those evaluating to integer value

(b) Real expression: those evaluating to a real (floating point) value

Thus, $3 + 5$ is an integral expression and $3.8 - 6.97$ is a real expression.

2. Logical or conditional expressions always result into either of the two values – true or false.

Thus $3 > 5$ and $x \leq 7$ are conditional expressions.

7.1 Data Types

One of the factors that determines the strength of a programming language is the number of different types of data that it can handle. In this regard, the C programming language is extremely strong. In a C program, almost every form of data can be interpreted and manipulated.

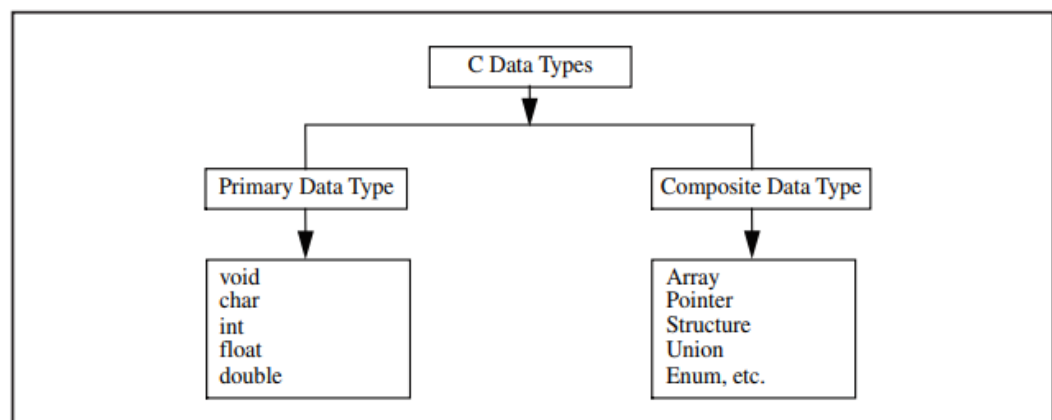
At the most basic level of a digital computer, all data and instructions are processed using only binary digits (0 and 1). As a result, the binary equivalent of the decimal number 65 is 0100 0001. Also stored is the character "A" as the binary equivalent of 65 (ASCII): 0100 0001. Both stored values are identical, but they represent different types of data.

Even if the value is only 0100 0001 as long as it is stored on the secondary storage unit, the meaning of a stored value is dependent on the type of variable in which the value is stored. If 0100 0001 is stored in an integer type variable, it will be interpreted as 65, while if it is stored in a character type variable, it will be interpreted as "A."

As a result, the data form refers to how a value stored in a variable is interpreted. In other words, a variable's data type refers to the type of data it may hold.

Every programming language supports a different set of data types. Furthermore, the size of data types (the number of bytes used to store a value) varies by language. Furthermore, it is hardware platform based.

C has a rich set of data types that is capable of catering to all the programming requirements of an application. The C-data types may be classified into two categories: Primary and Composite data types as shown in figure.



C has two distinct categories of data types – primary, and composite. Primary data types are the ones that are in-built with C language while composite data types allow the programmers to create their own data types.

There are five primary data types in C language.

1. `char`: stores a single character belonging to the defined character set of C language.



Example: `char a;`

2. `int`: stores signed integers. e.g., positive or negative integers.



Example: `int b;`

3. `float`: stores real numbers with single precision (precision of six digits after decimal points).



Example: `float c;`

4. **double**: stores real numbers with double precision, i.e., twice the storage space required by float.



Example: double d;

5. **void**: specify no values.

The following table shows the meaning and storage spaces required by various primary data types.

Data Type	Meaning	Storage Space	Format	Range of Values
char	A character	1 byte	%c	ASCII character set
int	An integer	2 bytes	%d	- 32768 to +32767
float	A single precision floating point number	4 bytes	%f	- 3.4*10 ³⁸ to + 3.4*10 ³⁸
double	A double precision floating point number	8 bytes	%lf	- 1.7 × 10 ³⁰⁸ to +1.7*10 ³⁰⁸
void	valueless or empty	0 byte	-	-

7.2 Additional Data Types

Primary C data types are available in a number of sizes, allowing for the storage of a broad range of values. A keyword (data type qualifier) is appended before the data type in a program to imply this.

For the int basic sort, data type qualifiers include short, long, signed, and unsigned. In C, you can specify an integer data type as a short int, int, unsigned int, or long int. The set of values and size of these qualifying data-types are determined by implementation. In contrast, short is smaller or equal to int, which is smaller than long. An unsigned int has a wider variety than a signed int because it does not store negative integers..

Also known as derived data types, composite data types are derived from the basic data types. They are five in number.

1. **Array**: Sequence of objects, all of which are of same types and have same name.



Example: int num [5];

Reserves a sequence of five locations of 2 bytes, each, for storing integers num[0], num[1], num[2], num[3] and num[4].

2. **Pointer**: Used to store the address of any memory location.



Example: int *ptr.



Note: - Asterisk (*) symbol is used to declare pointer variable.

3. **Structure**: Collection of variables of different types.



Example: A structure of employee's data, i.e., name, age and salary.

4. Union: Collection of variables of different types sharing common memory space.
5. Enumerated: Its members are the constants that are written as identifiers though data type they have signed integer values. These constants represent values that can be assigned to corresponding enumeration variables.

Enumeration may be defined as:

```
enum tag { member1, member2 .... member n};
```

E.g.: enum colors {red, green, blue, cyan};

```
colors foreground, background;
```

7.3 Operators

An operator is a symbol that instructs the machine to perform mathematical or logical operations on data held in variables. Data items (variables and constants) can be manipulated arithmetically in a C program using operators.

C operators can be classified into a number of categories.

- Unary operator
- Binary operator
- Arithmetic operators
- Relational operators
- Logical operators
- Conditional operators
- Assignment operators
- Bitwise operators
- Increment and decrement operators
- Special operators

Unary operator

Unary operators are operators that act upon a single operand to produce a new value.

There are following types unary operators in C

Unary Operator	Meaning
increment(++)	Increment value of variable by 1
decrement(- -)	Decrement value of variable by 1
sizeof()	Returns the size of its operand, in bytes.
Addressof operator(&)	Give an address of a variable.
NOT(!)	Used to reverse the logical state of its operand.
Unary minus (-)	Changes sign of its argument.

Binary operator

Binary operators are those that operate on two operands. For example, $a + b$ – the addition operator (+) surrounded by two operands – is a popular binary expression. Arithmetic, relational, logical, and assignment operators are all subsets of binary operators.

The binary operators are categorized as follow:

- Multiplicative operators: multiplication (*), remainder (%), and division (/)
- Additive operators: addition (+) and subtraction (-)
- Shift operators: left shift (<<) and right shift (>>)
- Relational operators: less than (<), less than or equal to (<=), greater than (>), and greater than or equal to (>=)
- Equality operators: equality (==) and inequality (!=)
- Bitwise operators: AND (&), OR (|), and XOR (^)
- Logical operators: AND (&&) and OR (||)

Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations include addition, subtraction, multiplication and division on one or more operands.

It is used to performed operations on numeric data types like int, float and double Arithmetic operators work on numeric type of operands. C provides all the basic arithmetic operators. There are five arithmetic operators in C.

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder after integer division

The division operator (/) requires the second operand to be non-zero, though the operands need not be integers. When an integer is divided by another integer, the result is also an integer. In such instances the division is termed as integer division. Consider the following:

```
int x;
```

```
x = 10;
```

What do you expect the value of $x/4$ to be? If you guessed 2.5, you are wrong.

The result is of course 2.5 however, since it is integer division (division operation in which both the operands are integers), the result 2.5 will be truncated to 2 to make the result an integer. In case you wish to get the correct value you make this division a float type as $x/4.0$.

The % operator is known as modulus operator. It produces the remainder after the division of two operands. The second operand must be non-zero.

Rest all the other operators work in their normal arithmetic way. Normal BODMAS rules are also applicable to these arithmetic operators.

7.4 Relational Operators

The relational operator compares two operands to see if they are equivalent, unequal, or if one is greater or inferior than the other.

The consequence is always a numeric value equal to true or false, regardless of whether the operands are variables, constants, or expressions. As previously stated, a non-zero result indicates

fact, while a zero result indicates false. Six relational operators are available in the C programming language.

=	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

A simple relation contains only one relational expression and takes the following form: <ae-1> <relational operator> <ae-2> <ae-1> and <ae-2> are arithmetic expressions, which may be constants, variables or combination of these. The value of the relational operator is either 1 or 0. If the relation is true, result is 1 otherwise it is 0.



Example:

Expressions	Result
6.3 <= 15	True
2.5 < -2	False
-10 >= 0	False
10 < 8+3	True

Logical Operators

Using logical operators, several relational expressions may be combined to create a single compound relational expression. To combine two or more relational statements, logical operators are used. Three logical operators are available in C. A compound relation behaves in the same way as a single relation generating a real or false value

Operator	Meaning	Result
&&	Logical AND	True if both the operands are true
	Logical or	True if both the operands are true
!	Logical not	True if the operand is false and vice versa



Example:

1. (age > 50 && weight < 80) will evaluate to true if age is more than 50 and also weight is less than 80. Otherwise it will evaluate to false.
2. (a < 0 || ch == 'a') will evaluate to true if a is less than 0 while ch is equal to 'a', false otherwise.
3. (!(a < 0)) will evaluate to true if a is greater than or equal to 0, false otherwise.

Assignment Operators

Assignment operators are used to store the result of an expression to a variable. The most commonly used assignment operator is (=). Be careful not to mistake assignment operator (=) for mathematical equality operator which is indicated by the same symbol.

An expression with assignment operator is of the following form:

`<identifier> = <expression>;`

When this statement is executed, `<expression>` is evaluated and the value is stored in the `<identifier>`.

Let us consider the following usage of assignment operator in C language.

```
int i;
```

```
i = 5;
```

```
i = i + 10;
```

The value now stored in the variable "i" will be 15. In this program, the current value stored in variable i is 5. Thus, while executing `i = i+10`, the right hand side will be evaluated to give a value 15. This value will then be assigned to the left hand side. As a result, the current value of i after execution of this statement will become 15.

C language provides a short cut to write arithmetic assignment expressions which takes the following form:

`<Variable> op = <expression>;`

This statement is identical to:

`<Variable> = <Variable> op <expression>;`

Thus, `i=i+3` can be written as `i+=3`

`Sum=sum-2` can be written as `sum-=2`

`i=i*5` can be written as `i*=5`

The advantages of using this form of assignment operators are:

1. The statement is more efficient and easier to read.
2. What appears on the L.H.S need not to be repeated and therefore it becomes easier to write for long variable names. Consider the following C code that illustrates this point.

```
int avery long variable name;
```

```
averylongvariablename = 2;
```

```
while (averylongvariablename < 20)
```

```
{
```

```
averylongvariablename*= averylongvariablename;
```

```
}
```

Increment and Decrement Operators

C has two very useful operators ++ and -- called increment and decrement operators respectively.

These are generally not found in other languages. These operators are unary operators as they require only one operand. The operands must be a variable name and not a constant.

The increment operator (++) adds one to the current value of the operand and stores the result back into the operand, while the decrement operator (--) subtracts one from the operand and stores the decremented value back into the operand.

There are two different forms of increment and decrement operators. When they are used before the operand, it is termed as prefix, while when used after the operand, they are termed as postfix operators.

**Example:**

```
int i = 5;
i++;
++i;
--i;
i--;
```

When used in an isolated C statement, both prefix and postfix operators have the same effect, but when they are used in expressions, each of them has a different effect.

In expressions, postfix operator uses the current value and then increments/decrements while in the prefix form the value is incremented/decremented first and then used in the expression. Consider the following examples:

E.g.: `b = a ++;`

this is postfix increment expression. This statement is equivalent to:

```
{b = a;
a = a+1;}
```

E.g. `b = -- a;`

this is prefix decrement expression. This statement is equivalent to:

```
{ a= a-1;
b = a; }
```

Consider the following C code that illustrates the usage of postfix and prefix increment operators.

```
int a = 10; b = 0; // a = 10 and b = 0
a++; // a = 11 and b = 0
b = ++a; // a = 12 and b = 12
b = a++; // a = 13 and b = 12
```

Conditional Operators

C provides a ternary operator called the conditional operator which is represented by `?:`. The syntax of this operator is given below.

`A?B:C`

Where “A” is a conditional expression resulting in either of the two values – true or false. The value generated by this operator in the expression depends on the value of the conditional expression “A”. If the value of “A” is true then the expression evaluates to “B” otherwise it results in “C”.

Bitwise Operators

You're aware that a numeric value is stored in binary form in a variable. Data is manipulated at the bit level using bitwise operators. These operators are used to measure bits or to move them left or

right. Only integer data types are supported by bitwise operators. The following is a list of the various bit wise operators available in the C language, along with their corresponding meanings.

Operator	Meaning
&	Bitwise Logical AND
	Bitwise Logical OR
^	Bitwise Logical XOR
<<	Left shift
>>	Right shift
~	One's complement

| (Bit-wise OR): Binary operator takes two operands of int type and performs bit-wise OR operation. With assumption that int size is 8-bits:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a | b yields [binary : 0000 1101]
```

& (Bit-wise AND): Binary operator takes two operands of int type and performs bit-wise AND operation. With same assumption on int size as above:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a & b yields [binary : 0000 0001]
```

^ (Bit-wise Logical XOR): XOR gives 1 if only one of the operand is 1 else 0. With same assumption on int size as above:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a ^ b yields [binary : 0000 1100]
```

<< (Shift left): This operator shifts the bits towards left padding the space with 0 by given integer times.

```
int a = 5; [binary : 0000 0101]
a << 3 yields [binary : 0010 1000]
```

>> (Shift right): This operator shifts the bits towards right padding the space with 0.

```
int a = 5; [binary : 0000 0101]
a >> 3 yields [binary : 0000 0000]
```

~ (one's complement operator): It is a unary operator that causes the bits of its operand to be inverted so that 1 becomes 0 and vice-versa. The operator must always precede the operand and must be integer type of all sizes. Assuming that int type is of 1 byte size:

```
int a = 5; [binary: 0000 0101]
~a; [binary: 1111 1010]
```

Special Operators

The C programming language also includes a variety of special operators that are not found in other languages. The comma operator, sizeof operator, pointer operators (& and *), and member selection operators (. and ->) are among these operators. When we introduce pointers, we'll talk about pointer operators, and when we talk about structures and unions, we'll talk about member selection operators.

We will discuss comma operator and sizeof operator in this section.

Comma Operator

This operator is used to link the related expressions together.



Example: `int x, y, z;`

`z = (x = 10, y=20, x+y);`

Here, the first statement will create three integer type variables – x, y and z. In the second statement, right-hand side will be evaluated first. Consequently, 10 will be stored in variable x, then 20 will be stored in variable y, and then values in x and y will be multiplied result of which will be stored in variable z. Thus, the value stored in the variable z will be 200 at the end of execution.

Size of Operator

The size of operator works on variables, constants and even on data types. It returns the number of bytes the operand occupies in the memory.

Consider the following C code for illustration.

```
sizeof(int); //Gives number of bytes occupied by an
//integer type variable
sizeof(float); //Gives number of bytes occupied by a
//float type variable
```

The output of this code will be 2, 4. Don't get disheartened if you get different result. This is only because the machine on which this program was run allotted 2 bytes for int type and 4 bytes for float type. The result that you get depends on the number of bytes allocated to these types on your machine. Nevertheless in all cases size of operator returns the number of bytes occupied by its operand on that particular machine.

7.5 Operator Precedence and Associativity

More than one operator can be used in a single sentence. The order in which operators are executed is determined by their precedence. When evaluating expressions involving multiple operators, precedence specifies the order in which operators should be added to the operands.

Depending on the level, operators of the same precedence are evaluated from left to right or right to left. This is referred to as an operator's associativity property. The following is a full list of operator precedence in the C language.

Description	Operators	Associativity
Function expression	()	Left→Right
Array expression	[]	Left→Right
Structure operator	.	Left→Right
Unary Minus	-	Right→Left
Increment/Decrement	++ --	Right→Left
One's complement	~	Right→Left
Negation	!	Right→Left
Address of	&	Right→Left
Value at address	*	Right→Left
Type cast	(type)	Right→Left
Size in bytes	sizeof	Right→Left
Multiplication	*	Left→Right
Division	/	Left→Right
Modulus	%	Left→Right
Addition	+	Left→Right
Subtraction	-	Left→Right
Left shift	<<	Left→Right
Right shift	>>	Left→Right
Less than	<	Left→Right
Less than or equal to	<=	Left→Right
Greater than	>	Left→Right
Greater than or equal to	>=	Left→Right
Equal to	==	Left→Right
Not equal to	!=	Left→Right
Bitwise AND	&	Left→Right
Bitwise XOR	^	Left→Right
Bitwise OR		Left→Right
Logical AND	&&	Left→Right
Logical OR		Left→Right
Conditional	?:	Right→Left
Assignment	=	Right→Left
	* = / = % =	Right→Left
	+ = - = &=	Right→Left
	^ = =	Right→Left
	<< = >> =	Right→Left
Comma	,	Right→Left

Summary

Precedence defines the sequence in which operators are to be applied on the operands, while evaluating the expressions involving more than one operator.

Keywords

Expression: A combination of identifiers and operators according to some rule that yields a value

Operator Preceding: The precedents of an operator determine the order in which expression will be evaluated

Operator: A symbol that works on one or more values to yield another value

The Size of Operator: The size of operator which is used to measure the data sizes. It a unary compile type operator that is to return the length of the variable or parenthesized type specifiers.

Self Assessment

1. Which one is not assignment operator?
 - A. +=
 - B. *=
 - C. &
 - D. >>=

2. (&=) operator is used for.....
 - A. Bitwise exclusive OR and assignment operator
 - B. Bitwise inclusive OR and assignment operator.
 - C. Bitwise AND assignment operator
 - D. None of these

3. Which one is Modulus AND assignment operator
 - A. <<=
 - B. %=
 - C. /=
 - D. None of these

4. In c which operators having highest priority
 - A. Logical operator
 - B. Relational operator
 - C. Arithmetic operator
 - D. Conditional operator

5. Conditional operator in c is also called
 - A. Relational operator
 - B. Logical operator
 - C. Ternary operator
 - D. None of these

6. Which are the data types
- A. Basic
 - B. Derived
 - C. Void
 - D. Above all
7. Which is not Derived Data Type
- A. Array
 - B. Pointer
 - C. Union
 - D. Char
8. Which is correct basic data type
- A. Int
 - B. Char
 - C. double
 - D. Above all
9. Find wrong statements
- A. `int a, b;`
 - B. `float rate;`
 - C. `int =2.5;`
 - D. `char = 'B';`
10. Which is not unary operator
- A. `++`
 - B. `--`
 - C. `&`
 - D. `+`
11. Relational operator is used with
- A. 1 operand.
 - B. 2 operands
 - C. Both 1 and 2
 - D. none of above
12. Which among the following is NOT a logical operator
- A. `&&`
 - B. `||`
 - C. `!`
 - D. `@`

Answers for Self Assessment

- | | | | | |
|-------|-------|------|------|-------|
| 1. C | 2. C | 3. B | 4. C | 5. C |
| 6. D | 7. D | 8. D | 9. C | 10. D |
| 11. B | 12. D | | | |

Review Questions

1. What are the different classes of operators available in C language?
2. Define the term "Expression". Explain the various types of expression in C.
3. What are the various logical and relational operators supported by C. Explain them with proper examples.
4. Draw a table that will provide a complete list of operators, their precedence level and their rules of association.
5. List down the advantages and limitations of using conditional operator in a C program.
6. Write short notes on:
 - (a) Shorthand assignment operators
 - (b) Bitwise operators
7. Write in detail about the various data type available in C.

**Further Readings**

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C